5 Skills to Build a Second Brain Like the 1%

The 5 free Claude skills that turn any folder into your AI operating system. Companion guide to the Ben AI YouTube video.

By **Ben van Sprundel** • benai.co • Version 3.0 • May 2026

How to use this guide

This is the long-form companion to the YouTube video. The video shows you what is possible in 15 minutes. This document gives you every step, every link, every template, and every limitation behind each of the 5 skills.

The guide is one chapter per skill, in the order you should run them:

1. `/os-setup` to bootstrap the vault.
2. `/os-operator` to keep it populated with real-time context.
3. `/os-optimizer` to keep it lean as it grows.
4. `/team-os` to share it across the team with permissions.
5. `/os-mcp` to make it run autonomously even when your laptop is closed.

If you only have 30 minutes today, do Skill 1 and stop. That alone is enough to call your vault set up. Everything after that compounds on top.

Install all 5 skills as a single plugin: the `obsidian` plugin inside the BenAI Suite. One install, all 5 commands available.

Table of contents

Why a second brain matters

A second brain gives any Claude session, on any platform (Cowork, Claude Code, Codex), persistent context and memory across every chat. Instead of starting each conversation with amnesia or copy-pasting context, your AI opens already knowing your business cold: your voice, your team, your strategy, your customers, your last 24 hours of work.

Four things make this layer special.

1. It is everything, not just facts. A second brain is not five "about me" docs. It is thousands of files: strategy, meeting transcripts, daily logs, departments, every YouTube transcript you ever made, team profiles, client context, decisions. Updated in real time as your business runs.

2. It is tool agnostic. The vault is a folder of markdown files on your computer. Obsidian is just a nice visual overlay on top. Any AI (Codex, Cowork, Claude Code, Gemini, ChatGPT desktop) can be pointed at the same folder and instantly get the full picture. No lock-in.

3. It is team shareable. Same vault, every teammate's AI agent. One person updates the brand voice, every channel inherits it. One person logs a decision, the whole team sees it.

4. The context compounds. This is the part most people miss. The value of the setup is not in the setup. It is in what builds inside it over time. Every decision logged, every correction saved, every project documented, every skill written. The AI you have at six months is not a slightly better version of day one. It is an entirely different tool.

This is also why the memory layer is the foundation for autonomous agents. Skills, routines, loops, Managed Agents, all the capabilities Anthropic keeps shipping, become 10x more useful the moment they can plug into real context about your business. Without the brain, they run on guesses. With the brain, they run on your operating reality.

> **[!important] The single most valuable thing you can do in AI right now is set up the memory layer. Not the latest model. Not the newest tool. The context that makes every model, every tool, and every agent actually useful for your specific business.**

The rest of this guide is the 5 skills that get you there.

Prerequisites

You do not need to be a developer. The whole system is plain markdown files in a folder, plus 5 skills we already built for you. If you can install an app and edit a text file, you can do this.

ITEM	WHY	COST
Obsidian (free)	The visual overlay for reading and editing your vault	$0
Claude Pro or Max	The first run of `/os-setup` and `/os-optimizer` burn through credits	$20+/mo
Claude Cowork or Claude Code	The AI side that reads and writes your vault	included with Pro/Max
A stable folder location on disk	Once you commit, do not move it	$0
The `obsidian` plugin from BenAI Suite	One install, all 5 skills	$0

For Skill 4 (Team OS) you also need a free Relay.md account. For Skill 5 (OS MCP) you also need a free or paid Railway account ($0 to ~$25/mo).

The rule from here on: every Claude session you start, the working directory is the vault folder. Always. If you forget this, Claude has no access to your context and the system feels broken when it is not.

Skill 1. /os-setup. Bootstrap the vault

The goal of Skill 1 is to end the day with a vault that is structured, has your initial context populated, and is wired to Claude. After this skill, every new Cowork or Claude Code session opens already knowing your business.

This skill solves three things that people get wrong when they try to set this up by hand.

1. **Context population.** What context do you actually put in there?
2. **Folder structure.** How should it be organized so it scales without becoming a mess?
3. `CLAUDE.md` **and index files.** How does Claude know where to find and store what?

The mindset before you start

Start simple. The point of day one is not a perfect vault with thousands of files. It is a small, structured starting vault that you actually use. Mine started with 30 or 40 documents. Six weeks later it had hundreds. Today it has thousands. It grows naturally once you live in it.

Do not try to migrate 5 years of Notion exports on day one. Do not stress about the perfect folder name. Pick the defaults the skill suggests and move on.

Run the skill

1. Pick or create a folder on your computer. Anywhere stable. Avoid iCloud Drive if you can (sync conflicts).
2. Open Claude Cowork or Claude Code with that folder as the working directory.
3. Type `/os-setup`.

The skill runs in three phases.

Phase 0: pick your mode

The first question is whether you are a solopreneur or professional, or a business with a team.

- **Solo / Professional** gives you a leaner folder structure: Context, Daily, Projects, Resources, Skills, Intelligence.
- **Business / Teams** adds: Departments (for SOPs by department), Team (one folder per person with their profile, daily notes, task list), and Onboarding (for new hires).

Most folks pick Solo on day one even if they have a small team. You can always re-run the skill later to switch.

Phase A: bootstrap the structure

The skill creates the folders, writes the system files (root `CLAUDE.md`, per-folder `CLAUDE.md` files, Obsidian config, hooks, output styles, an initial daily note), and sets up the memory layout. You watch this happen. It takes about 30 seconds.

The structure looks like this (Solo mode):

```
MyBrain/
├── CLAUDE.md
├── Context/
│   ├── brand.md
│   ├── icp.md
│   ├── strategy.md
│   ├── team.md
│   └── CLAUDE.md
├── Daily/
│   └── 2026-05-12.md
├── Intelligence/
│   └── CLAUDE.md
├── Projects/
│   └── CLAUDE.md
├── Resources/
│   └── CLAUDE.md
└── Skills/
    └── CLAUDE.md
```

Every folder has its own `CLAUDE.md` that tells Claude what lives there and how to navigate it. The root `CLAUDE.md` has the routing table, voice rules, and anti-patterns. This is the file that loads on every session.

Phase B: the onboarding interview

The skill now interviews you. It asks 10 questions to populate the initial Context files. The framing that works best: imagine you are hiring an extremely smart replacement for your own role. You have one folder of docs to hand them. What goes in?

The 10 areas the skill covers:

1. **You.** Role, background, working style, how you make decisions.
2. **The business.** What it does, why it exists, how it is structured, its history.
3. **The offer.** What you sell, how it is priced, the value it delivers.
4. **The customer.** Who they are, what they need, how they think and buy.
5. **The voice and brand.** How you sound, your positioning, what you stand for.
6. **The strategy.** Where you are heading, your priorities, your key metrics.
7. **Your day-to-day work.** Calls, writing, building, reviewing, planning. Recurring rituals.
8. **The team and people.** Who is on the team, their roles, key external relationships.

9. **The market.** Industry, competitors, trends.
10. **The operations.** Tools, processes, SOPs, how work actually gets done.

Take time here. As my co-founder likes to say: order a pizza, grab a beer, sit down one night with Claude and just do this properly. Use an audio transcription tool like Wispr Flow to brain-dump. Drop in existing docs from Google Drive or Notion if you have them. It does not need to be structured. The skill will structure it and route every piece to the right Context file.

What you have when it finishes

- `Context/` populated with your real `brand.md`, `icp.md`, `strategy.md`, `team.md`, plus whatever else your interview surfaced.
- A root `CLAUDE.md` with the routing table, rules, and anti-patterns.
- Per-folder `CLAUDE.md` and `index.md` stubs.
- One daily note for today.
- A working folder Claude can read on every session.

The first run uses more credits than a normal session because the skill is generating a lot of context in one pass. This is expected. Plan to do it during a session where you are not racing a deadline. After day one, normal usage is unremarkable.

Install Obsidian and point it at the folder

Managing this folder from Finder or Explorer is not intuitive. Install Obsidian (free, **obsidian.md**), open it, click "Open folder as vault," point at the folder you just created. Done. Obsidian instantly visualizes the folder as a graph and gives you a real editor.

You do not need any Obsidian plugins on day one. Skills 4 and 5 will install the ones you actually need.

Day-one checklist

Before you move on:

- [ ] 6 folders exist (or 8 for Business mode).
- [ ] Root `CLAUDE.md` exists with a routing table.
- [ ] At least one Context file is filled with real content (not template placeholders).
- [ ] One daily note exists.
- [ ] You can run a Claude session in the vault folder and Claude reads `CLAUDE.md` on the first message.

Your first conversation

Open Cowork or Claude Code in the vault. Try these:

1. **"Who am I?"** Claude should respond with details from your Context files. If it cannot, the routing is not working.
2. **"What did we ship yesterday?"** Claude reads the latest daily file and reports.
3. **"Draft a LinkedIn post about [topic]."** The post should sound like you, not generic. If it sounds generic, your `Context/brand.md` needs more specificity.
4. **"Add a task to follow up with [person] tomorrow."** Claude writes to your task list.

If those four work, the foundation is solid. If they do not, fix the foundation before moving to Skill 2.

Migrating from Notion, Roam, or OneNote

You probably already have a knowledge base somewhere. Here is the 30-second migration:

- **From Notion:** Settings > Workspace > Export content > Markdown & CSV. Drop the unzipped folders into your vault. Some block types translate poorly (databases become CSVs, embeds break). Do not waste time hand-cleaning before importing. Skill 3 will lint it.
- **From Roam Research:** Settings > Export All > Markdown. Same drop-in process.
- **From OneNote:** harder. Use OneNote2MD or copy section by section.
- **From Apple Notes / Google Docs:** copy-paste section by section, or export to PDF and OCR. Honestly, often easier to just summarize the most important notes by hand than to migrate everything. Old notes you have not opened in 6 months are not worth migrating.

Keep feeding it

The most important habit after the setup is conscious of saving meaningful work to the vault. Claude will do some of this automatically when it judges something important. But for any real decision, framework, customer insight, or lesson, tell Claude to save it. That is how the brain grows.

Skill 2. /os-operator. Real-time context on a schedule

You have a vault. It has your initial context. The problem is that the vault has no idea what happened today. The meeting you just had, the Slack thread your co-founder tagged you in, the question that landed in your community. Without something pulling that in, the vault goes stale within a week.

The `/os-operator` skill builds and schedules a personalized **Operator** prompt: a recurring task that pulls real-time context from your tools, updates the vault, and flags what needs human eyes.

What the Operator does

Three jobs, on every run:

1. **Populate.** Pull from outside sources and write to the right vault folder.
   - **Meetings**: Fireflies, Otter, Granola, Read.ai transcripts. Routes to `Intelligence/meetings/{type}/{date-title}.md`. Wikilinks attendees to their `Team/{name}.md` profiles. Pushes action items into each person's task list.
   - **Chat**: Slack, Teams, Discord, Telegram. Daily summaries of DMs and channels you care about. Tags appended to the right person's daily note.
   - **Community**: Circle, Discourse. Escalates urgent posts.
   - **Cowork session logs**: appends a session summary to today's daily note.
   - **Any other connector** you have wired up in Claude. The Operator auto-detects what is available and asks which to use.
2. **Housekeep.** Roll yesterday's daily forward. Archive completed projects. Clean up stubs. Light dedupe. (The heavy lifting is Skill 3.)
3. **Escalate.** If anything looks urgent (a customer churn signal, a Slack tag from someone important, a sync failure), it sends a DM. You decide who to: yourself, your co-founder, an ops lead. It does not auto-fix urgent things, it surfaces them.

How the skill builds your Operator

The skill is interview-driven, but it tries to interview you as little as possible. Here is the flow:

1. **It reads `Context/` and `CLAUDE.md` first.** Silently. Pulls out your org name, team scope, brand voice, folder layout, conventions. This means it does not re-ask things you already told `/os-setup`.
2. **It probes your live connectors.** It does not just ask which tools you use. It checks which connectors are actually wired up to Claude and confirms each one responds.

3. **It asks only the gaps:**
   - How often should this run? (Once a day is the recommended default. Some teams run twice. More than that just burns credits.)
   - Which detected connectors should it use?
   - Who should it DM if it flags something urgent?
   - What is your nightly token budget cap?
   - How should it sign its reports?
4. **It renders and saves the prompt locally** in your vault.
5. **It schedules it automatically.** It calls the `schedule` skill internally and wires the recurring trigger. You do not need to run a separate `/schedule` command.

A real nightly run, mine

Here is one night of my Operator, from a recent log:

```
03:00 — Pulled 4 Fireflies transcripts → wrote meeting notes,
        wikilinked attendees, pushed 11 action items to 4 people
03:08 — Pulled 24 Slack threads I was tagged in → appended summaries
03:09 — Pulled 6 Circle community posts → flagged 2 for response
03:10 — Appended session summary from yesterday's Cowork logs
        to Daily/2026-05-11.md
03:11 — Rolled forward incomplete tasks from yesterday's daily
03:12 — Posted 1 escalation to #vault-flags (customer reply needed)
03:14 — Wrote nightly report to Daily/2026-05-12.md
```

By 8 AM the vault is current. Three minutes of human review (read the report, act on the escalation) replaces what used to be an hour of catching up on the day before.

A note on what to schedule for yourself

Once your Operator is running, scheduled tasks for yourself become powerful. Morning briefings are the obvious one. When you open Claude in the morning, a separate scheduled task can read the Operator's nightly report, your daily note, your task list, and tell you the three priorities for today plus anything important that happened while you were away. Naveed runs one of these. So do I.

You can build these yourself in one prompt now that the vault has real context to read.

Limitations to be honest about

- **Your laptop must be open for the scheduled run to fire.** Claude Desktop scheduling is local. This is what Skill 5 (`/os-mcp`) solves: it moves the Operator to the cloud so it runs even when your

laptop is closed.

- **It fires on a clock, not on events.** It will not react instantly to a Slack message landing right now. Only at the next scheduled run.
- **It burns credits even on quiet days.** A lint and a connector poll cost the same whether something happened or not. The token cap is there for this reason.
- **First two weeks: read every report.** This is how you learn whether your Operator prompt is tuned correctly. After two weeks, you trust the routine ones and only read the flagged ones.

The honest sequence

Do not jump straight to autonomous nightly runs. Build trust first:

1. Run the skill once and read what it generates.
2. Let it run on a schedule, manually inspect the output for a week.
3. Once you trust it, let it run in the background and only check the escalations.
4. Once you trust the schedule, move it to autonomous cloud runs (Skill 5).

Skill 3. /os-optimizer. Keep it lean as it grows

The vault grows fast once you start using it. Daily notes pile up. Meeting transcripts land every day. Skills get added. Folders accumulate stuff that should have been routed somewhere else. Wikilinks die when files get renamed. Two files start saying contradictory things about the same customer.

This is context bloat. Bloat costs you in four ways:

1. **Tokens.** A bloated `CLAUDE.md` loads on every session. Every session.
2. **Speed.** More tokens means slower model responses, even before the context window fills.
3. **Accuracy.** Context rot is real (Chroma's research). Even before you hit the limit, more irrelevant tokens make the model perform worse on the relevant ones.
4. **Trust.** When two files contradict each other, your AI picks one at random. You stop trusting the outputs.

`/os-optimizer` is the maintenance skill that fixes all of this. Run it weekly or bi-weekly.

What it audits

The skill applies 9 frameworks to every markdown file in the vault. You do not need to know all 9, but they exist so you know the audit is grounded, not vibes:

- **F1. Anthropic `CLAUDE.md` best practices.** Length, specificity, pruning test ("would removing this line make Claude mess up?").
- **F2. Karpathy LLM Wiki pattern.** Schema-then-detail. Root file is the index. Folder files are the wiki pages. The nightly lint is the third leg.
- **F3. Caveman compression.** Strip filler, hedging, and pleasantries from agent-facing files. Use symbols where they replace prose.
- **F4. Chroma context rot research.** Smaller high-signal context outperforms large vague context, even when the window is not full.
- **F5. Anthropic Memory architecture.** Files at `/mnt/memory`, not vectors. Per-folder discovery, not whole-tree pre-loading.
- **F6. Progressive disclosure.** Root file is small. Folder-level details load on demand.
- **G7. Hygiene.** Frontmatter compliance, em dash detection, duplicate H1s, broken wikilinks.
- **F8. Reflection (Anthropic Dreams).** Promote durable knowledge out of daily logs and meeting notes into Context or Resources. Lint contradictions across files.
- **F9. Architecture and Discoverability.** Walks the actual co-worker-Claude discovery chain: root `CLAUDE.md` → routing → folder `Plot.md` (or `index.md`) → file. Audits whether the routing table

tells the truth about what is actually in the folder. Flags files that are unreachable in 3 hops. Proposes reorganizations grounded in your stated Context, not generic best practice.

What it actually does

When you run `/os-optimizer`, it goes through your vault file by file and produces fixes for every issue it finds. The categories you will see in the dashboard:

1. **Agent-facing file compression.** Compresses `CLAUDE.md`, `SKILL.md`, and other instruction-layer files. My root went from ~1,900 tokens to ~1,400 tokens. That saves ~500 tokens on every conversation, forever.
2. **Navigation fixes.** Verifies every routing entry resolves to a real folder. Rewrites routing descriptions when they no longer match the folder. Generates missing folder index files. Refreshes stale ones against actual contents. Flags unreachable files.
3. **Knowledge layer cleanup.** Resolves contradictions across files. Merges duplicates (3+ files with 60%+ overlap) into one canonical entry, redirects wikilinks, archives the originals. Promotes durable knowledge from daily logs to Context. Updates stale Context entries against recent decisions, preserves prior wording in a `## History` section.
4. **Folder structure improvements.** Spots misplaced files. Detects folders doing the same job. Proposes 1 to 3 structural reorganizations grounded in your Context, with migration plans.
5. **Hygiene.** Adds missing `status:` and `tags:` frontmatter. Removes H1s that duplicate the filename. Replaces em dashes. Brings READMEs and indexes into compliance.

The operating philosophy

This is the part most "vault optimizer" tools get wrong. Three rules:

1. **Every finding ships a concrete fix.** No flag-only. No "manual review pile." When you run apply-mode, every finding becomes either an applied edit, a saved migration step in a dated reorg plan, or a recorded decline. Nothing lingers as an open warning across runs.
2. **Walk per item for anything semantic.** Mechanical fixes (em dashes, duplicate H1) bulk-apply. Anything semantic (wikilink repointing, merges, routing rewrites, reorganizations) is walk-only. You confirm the destination, the winner, or the wording per item.
3. **Apply now or save to plan.** For high-blast-radius items, you pick per finding. Small fixes default to apply now.

What the dashboard shows

The skill saves a single HTML report grouped by framework. It does not inline the HTML in chat. You get a path and a one-paragraph summary. Open the report in your browser to see:

- Tokens before vs after, per session and annualized.
- Score climbing (e.g., 67 → 84 (+17), "Bloated" → "Tuned").
- Per-section savings tables.
- A per-framework list of what was found, what was fixed, and what was saved to plan.

A real example from my last audit: 247 auto-fixes applied, ~7,000 findings flagged for walk-review across the rest of the week. Your score does not jump to 100 in one run. It climbs over time.

Cadence

Weekly is the sweet spot if you are using the vault daily. Bi-weekly if you are a lighter user. Less than that and bloat accumulates. More than that and you are spending credits on nothing.

You can also wire `/os-optimizer` into a routine in Skill 5 so it runs autonomously on a weekly cadence in the cloud.

Skill 4. /team-os. Share the vault across your team

Once you have lived in the vault for a few weeks and trust the system, the next move is to give your team the same brain. This is where it stops being a personal productivity tool and starts being your operating system.

The moment a second person joins, you need three things the vault does not give you out of the box:

1. **Real-time sync.** My edit shows up in your Obsidian and your Cowork session in seconds, not on the next git pull.
2. **Permissions.** Who can read what, who can write what.
3. **Deletion safety.** A teammate's local delete must not propagate to everyone else's vault.

`/team-os` solves all three.

Why the obvious options fail

Before getting to the solution, here is why most teams try the wrong thing first.

- **GitHub.** Not real time. You need to git pull. No rich-text UX. Devs only.
- **Google Drive.** Not version-controlled in any usable way. Hard to diff. Pulls from Claude need MCP, which adds a layer of cost and latency.
- **Notion.** Same MCP problem, plus locked-in formatting that does not pipe cleanly into an LLM.
- **iCloud or Dropbox.** Works for one person across their own devices. For a team, no real-time sync, no permissions, no deletion safety.
- **Obsidian Sync (official).** Works, but no real-time, no permissions.

The stack: Relay + the BenAI fork

Two plugins, different jobs.

Relay (official, by System 3). Real-time CRDT sync. This is the layer that gets your edits to teammates in 2 seconds with no merge conflicts even when two people edit the same file at the same moment. Built on Yjs. Install it from Obsidian's Community Plugins by searching "Relay." Free tier works for small teams.

BenAI Relay fork. Adds what the official plugin does not have: per-user read and write permissions, folder-level access control, and deletion safety. This is the layer that makes "team OS" possible without an admin nuking your strategy doc by mistake.

> [!important] Relay handles "everyone sees the latest." Our fork handles "not everyone should see everything."

What you can lock down with the fork

- **Read and write per user.** Aryan can read `Context/`, only Ben can write to it.
- **Folder-level access control.** `Departments/Finance/` is invisible to anyone who is not Oskar or Ben.
- **Deletion safety.** If someone deletes a file locally, the deletion does not propagate to the server. Non-admins literally cannot wipe the vault for everyone else.

What the skill does

The `/team-os` skill replaces the official Relay plugin (`system3-relay`) with the BenAI fork (`benai-relay-fork`). The compiled fork ships inside the skill, so no manual download.

You give it one thing: your Obsidian vault folder. If you run the skill from inside the vault, it auto-detects. Otherwise it walks you through finding it via Obsidian's Settings > About > Show vault folder.

The skill:

1. Asks you to close Obsidian.
2. Removes the upstream Relay plugin if present.
3. Installs the BenAI fork (three files: `main.js`, `manifest.json`, `styles.css`).
4. Updates `community-plugins.json` automatically so Obsidian picks it up.
5. Tells you to reopen Obsidian.

When you reopen, you have a new plugin pane with permission controls. You can mark documents and folders as private, set read-only on strategy docs, give edit access to specific teammates.

Per-teammate install

Each teammate runs the skill once on their own machine. There is no server to deploy, no Railway, no DevOps. The fork is a plugin, not a backend. The Relay sync layer is what carries everyone's edits between machines.

Trade-off: every teammate needs a free Relay.md account. That is the one third-party dependency.

When this works and when it does not

Obsidian + Relay + the fork works great up to about 10,000 notes and 50 teammates. Past that, you graduate to a different stack (Postgres + R2 for storage, custom permissions service). Most teams will never hit this. If you do, **book a call**.

Permissions are folder-level, not field-level. You cannot say "this person sees rows 1 to 50 of this file but not rows 51 to 100." Whole files, not partial reads.

When to skip Skill 4

If you are a solo founder, freelancer, or consultant with no teammates yet, skip this. The rest of the system works perfectly without it. Come back when you have someone to share with.

Skill 5. /os-mcp. Run it autonomously, laptop closed

Everything up to here runs locally. Your laptop has to be open and Claude has to be running for the Operator and the Optimizer to fire. That is fine for the first few weeks. Eventually you want this stack running autonomously even when your laptop is closed.

This is what routines and Managed Agents are for. They run in the cloud on Anthropic's infrastructure. They do not need your laptop. The only thing missing is: how does a cloud agent read and write files that live on your computer?

`/os-mcp` solves it. It wraps your Relay-synced vault in a Model Context Protocol server and deploys that server to your own Railway account. The cloud agent talks to the MCP. The MCP talks to Relay. Relay holds the real-time copy of your vault.

The full picture

```
Your laptop's vault folder
      ↕  (Relay real-time sync, from Skill 4)
Relay.md cloud
      ↕  (Relay MCP v2, deployed by Skill 5)
Your Railway-hosted MCP server
      ↕  (MCP protocol)
Anthropic Managed Agent or routine
```

Once this is in place:

- The Operator from Skill 2 can run as a cloud routine, every night, with your laptop closed.
- The Optimizer from Skill 3 can run as a cloud routine, once a week, with your laptop closed.
- Any other automation that needs vault context (daily briefings, morning standups, customer-reply triage) can be wired up the same way.

What `/os-mcp` does

The skill deploys the bundled Relay MCP v2 server to your own Railway account. The MCP source ships inside the skill, no separate repo to clone.

You provide one thing: a Railway account token. You create it at **railway.com/account/tokens** in 30 seconds.

The skill:

1. Reads the token.
2. Provisions a Railway service from the bundled source.
3. Deploys.
4. Returns the URL of your MCP server.
5. Walks you through adding the MCP to Claude (Cowork, Claude Code, or claude.ai web).

After deploy, you sign in to the MCP with your Relay.md credentials (OAuth). The MCP then auto-discovers your relay vault and the folders inside it through PocketBase. No vault GUIDs. No folder maps. No config.

Why Skill 4 has to come first

The MCP talks to your vault through the Relay protocol. If you do not have Relay installed (Skill 4), the MCP has nothing to talk to. Skill 4 is the prerequisite for Skill 5. This is also why solo users can still benefit from Skill 4 even without teammates: it is the bridge that makes Skill 5 possible.

Cost

Railway free tier covers small vaults and light routines. Heavier usage runs $5 to $25 a month. The MCP is yours, deployed in your account, you control it.

What you wire up next

Once your MCP is live, you set up two routines in Claude:

1. **Nightly Operator routine.** Same prompt as your local Operator from Skill 2, but scheduled as a Claude routine (Managed Agent) instead of a local scheduled task. Runs at 02:00 every night with your laptop closed.
2. **Weekly Optimizer routine.** Same as `/os-optimizer` but on a Sunday-night cron. Wakes up, lints the vault, saves the dashboard to `Intelligence/decisions/`, sends you a Slack DM with the summary.

You can add more routines over time. Daily briefings. Inbox triage. Customer-call summaries. Anything that needs vault context to do well.

Adoption reality

Even with all 5 skills set up perfectly, the hardest part is still getting your team to use it.

The honest line, from Naveed, our head of operations:

> **[!important] "Adoption of a second brain across your team is not straightforward, and it takes some time."**

Top-down rollouts on tools like this die fast. Every founder who has tried "everyone log your work in Notion by Friday" knows what happens. Compliance for a week, then nothing.

How to actually get adoption

1. Run it solo for 2 to 3 weeks before bringing anyone in. You will find friction your team will not have to deal with. Naming conventions you regret. Folders that turned out wrong. Routines that fired too often. Fix all of that alone first. Bring teammates a working system, not a half-built one.

2. Add the most curious teammate next, not the most senior. They become the internal champion. The senior person can come later. The curious one will iterate with you. The senior one will point out problems and wait for you to fix them.

3. Roll out across the team organically. As the curious teammate uses it visibly (better drafts, faster meeting summaries, fewer "where is that doc"), other teammates ask how. Onboard one at a time. Do not announce it at an all-hands.

The compounding loop

Every teammate who adopts feeds the brain richer context. Richer context makes the brain more useful. More usefulness pulls the next teammate in.

> **[!tip] The first month is a slog. By month three the system pulls people in on its own.**

Plan for the slog. The compounding only kicks in once you push past it.

Honest limitations

This system is not magic. Here is what does not work yet or never will.

Second brain vs. database

A second brain is for unstructured knowledge: ideas, decisions, context, voice, history. A database is for structured data: customer records, inventory, ledger entries, anything you query by field.

> **[!warning] If you need a database, use a database. If you need a brain, use markdown.**

Trying to put 50,000 customer records in markdown files will not work. Trying to capture "what did we decide and why" in a Postgres table will not work either. Use both, for different jobs.

First-time setup will burn through your Claude budget

The `/os-setup` skill burns more credits than a normal session on its first run, because it is generating all your Context files at once. Plan accordingly. After day one, normal usage is normal cost.

Obsidian cannot render HTML, PDFs, or non-markdown natively

Drop a PDF in your vault and Obsidian shows you a link, not a preview. Same with HTML. Workarounds: keep visual artifacts in a separate `assets/` folder and view them in Preview, or install the Annotator and HTML viewer plugins.

No native vector or semantic search

Obsidian's search is keyword-based. It does not understand "find notes similar to this one." For our scale (a few thousand notes per team), the agent navigates via wikilinks and folder structure, which works. If you grow into hundreds of thousands of notes, you will want a vector layer (Smart Connections, Quartz, or a custom embedding pipeline).

No native Claude-in-Obsidian connector

There is no official "Claude inside Obsidian" plugin. Claude reads your vault from Cowork or Claude Code, not from inside Obsidian. The workflow is: write in Obsidian, prompt in Claude. Some community plugins (Claude in Obsidian, AI Helper) bring Claude into the editor; we do not endorse one yet.

Local schedules require something to be on

Claude Desktop scheduling requires Claude Desktop to be open. This is exactly what Skill 5 fixes by moving runs to the cloud.

Permissions are folder-level

The BenAI Relay fork gives you per-folder permissions. It does not yet give you per-section or per-line permissions. If you need "this person sees rows 1 to 50 but not rows 51 to 100," you need a different tool.

Why you should start today

The compounding-context moat

Every meeting, decision, correction, and customer note your team logs today is institutional memory you can hire against later. New hires onboard against the vault, not against tribal knowledge. Decisions reference past decisions, not Slack archaeology.

A competitor who copies your structure in 6 months still has a 6-month context deficit. Your vault has 6 months of meetings, decisions, customer voice, and corrections. Theirs has none.

> **[!important] Every meeting, decision, correction logged today is a 6-month head start over a competitor who copies the structure later.**

This is the moat. Not the folder structure. The folder structure is public (this guide is the structure). The moat is the months of compounding context inside it.

Start today vs. next quarter

The setup is 30 minutes. Even if you only run Skill 1 today and never get to Skill 2, the marginal value is positive. You have a vault. You have one daily note. Claude reads your context. That is already better than starting fresh every session.

Every week you delay is a week of decisions, customer voice, and meeting notes that did not get captured. You cannot retroactively log them.

What good looks like at 6 months

At the 6-month mark, a team that runs this well looks like:

- 1,000 to 5,000 markdown files in the vault.
- A daily note for every working day, populated by the Operator.
- 20 to 50 skills, all pointing at vault Context.
- New hires productive in under a week (because they read the vault to onboard).
- Brand voice consistency across every channel (because every channel's skill reads `Context/brand.md`).
- Meeting follow-ups that actually happen (because action items land in task lists automatically).
- Routines running nightly and weekly in the cloud, with your laptop closed.

That is the destination. This guide and these 5 skills are the path.

Appendix A. All resource links

External research and frameworks

- Anthropic CLAUDE.md best practices: **code.claude.com/docs/en/best-practices**
- Karpathy's LLM Wiki gist: **gist.github.com/karpathy/442a6bf555914893e9891c11519de94f**
- Caveman compression framework: **github.com/JuliusBrussee/caveman**
- Chroma's context rot research: **research.trychroma.com/context-rot**
- Anthropic Managed Agents Memory architecture: see **docs.claude.com**

Tools

- Obsidian: **obsidian.md**
- Relay (official): **relay.md**
- Railway: **railway.app**
- Railway account tokens: **railway.com/account/tokens**

Ben AI

- The 5-skill bundle: **benai.co/second-brain-bundle**
- YouTube channel: **youtube.com/@benvansprundel**
- Ben AI Accelerator (community + done-for-you): **benai.co/accelerator**
- Custom Business OS (we set it up for your team): **book.benai.co**
- Twitter / X: **@benvansprundel**

Appendix B. Templates you can copy

These are the same templates `/os-setup` will generate for you. Here so you can see what they look like or build by hand if you prefer.

Root `CLAUDE.md` template

```
# {Your Org} Operating System

Org AI assistant. Vault = Obsidian knowledge base AND operating system.
All state lives in markdown files you read, write, and maintain.

## Session Startup

On first response:
1. Silently read latest `Daily/` file for org context.
2. Ask: "Who is this session for?" to identify active profile.
3. Silently read `Team/{name}/{Name}.md` + latest entry in
   `Team/{name}/Daily/`.

Active profile = where session output is written.

## Knowledge Routing

| Type | Route to |
|---|---|
| Person preferences, daily notes, tasks | `Team/{name}/` |
| Strategy, OKRs, brand, stakeholders | `Context/` |
| Project info | `Projects/{category}/{name}/` |
| Department info, SOPs | `Departments/{name}/` |
| Meetings, competitors, decisions | `Intelligence/` |
| Reusable content | `Resources/` |
| Tasks, action items | active profile's `task-list/` |

## Rules

1. Meaningful work goes to `Team/{name}/Daily/YYYY-MM-DD.md`. Never root.
2. Use `[[wikilinks]]` for every entity in vault files.
3. Every note: standalone, composable, lego block.
4. Use callouts (`> [!type]`) for visual structure.
5. Auto-save to the right vault folder. Never ask permission.
6. Never use em dashes.
7. Never create files in vault root.

## Anti-Patterns

- Do not duplicate the filename as a `# Title` heading.
- Do not create orphan notes.
- Do not update vault files on casual chat.
```

Per-folder `CLAUDE.md` template (example for `Projects/`)

```
# Projects/

## Layout

Each project has its own folder. Day one = just a README.md.
Subfolders appear as content arrives:
- `research/`: competitor analysis, market notes
- `specs/`: requirements, briefs
- `drafts/`: written content, scripts, outlines
- `notes/`: working scratch notes
- `feedback/`: review comments, reactions

## Rules

- Subfolders appear on demand, do not pre-create.
- README.md is the index, not the content dump.
- Completed projects go to `Intelligence/archive/{name}/`.
- Always include `project:` in note frontmatter.
```

`Context/operator.md` starter template

```
---
type: role
status: active
tags: [operator, agent, role]
---

# Vault Operator

## Role

The autonomous agent that owns the vault. Runs nightly and on events.
Reports to the Champion (Ben).

## Responsibilities

### 1. Auto-update (nightly 02:00)
- Pull Fireflies transcripts from last 24h.
  Write to `Intelligence/meetings/{type}/{YYYY-MM-DD-title}.md`.
- Pull Slack threads where any teammate was tagged.
  Append summaries to that teammate's daily note.
- Pull Cowork session logs from last 24h.
  Append summary at the bottom of the company daily note.

### 2. Lint (nightly 03:00)
- Scan for duplicate files (same content or same purpose).
- Scan for dead `[[wikilinks]]`.
- Scan for skills referencing files that no longer exist.
- Scan for contradicting Context files.
- Post findings to `#vault-flags`. Do NOT auto-fix.

### 3. Auto-research (on demand)
- When a session asks a question with no vault answer,
  flag it, propose an addition, wait for human approval.
- Never write to `Context/` without approval.

## Voice

Direct. Specific. No hedging.
Every report cites file paths and line numbers.

## Hard rules

- Never delete a file.
- Never write to `Context/` without human approval.
- Never run during business hours (09:00 to 18:00).
- Budget cap: 50,000 tokens per nightly run. Stop and flag if exceeded.
```

Sample daily note template

```
---
type: daily
date: 2026-05-12
status: active
tags: [daily, log]
---

## What happened today

(populated by the Operator from Fireflies + Slack + Cowork logs)

## Decisions

(populated manually or via /log-decision)

## Wins

## Blockers

## Tomorrow
```

Sample project README template

```
---
type: project
status: active
project: {name}
date: 2026-05-12
tags: [project, {department}]
---

# {Project Name}

## Goal

(one sentence: what success looks like)

## Status

(active / on-hold / completed)

## Owner

[[{Name}]]

## Stakeholders

[[{Name}]], [[{Name}]]

## Linked OKR

`Context/strategy.md#okr-{id}`

## Next steps

- [ ] {step}
- [ ] {step}

## Related

- Research: `research/`
- Specs: `specs/`
- Drafts: `drafts/`
```

Appendix C. The skill bundle

All 5 skills ship inside one plugin: the `obsidian` **plugin** in the BenAI Suite. One install, all 5 commands.

SKILL	COMMAND	WHAT IT DOES
OS Setup	`/os-setup`	Bootstrap vault structure + 10-question onboarding interview
OS Operator	`/os-operator`	Build and schedule a personalized Operator for real-time context
OS Optimizer	`/os-optimizer`	9-framework audit and clean-up of the vault
Team OS	`/team-os`	Replace Relay plugin with BenAI fork (RBAC, deletion safety)
OS MCP	`/os-mcp`	Deploy Relay MCP v2 to your Railway for autonomous cloud runs

Download the bundle: **benai.co/second-brain-bundle**

Accelerator-only

The full skill library (50+ skills, kept current as Claude evolves) is inside the Ben AI Accelerator. Includes channel-specific skills (`linkedin-writer-vault`, `youtube-outliner`, `newsletter-writer`, `dm-responder`, `cold-email`), meeting and intel skills (`meeting-router`, `slack-summarizer`, `lead-intelligence`), and builders (`routine-creator`, `operator-builder`). Every skill points at vault Context, so they all inherit your brand voice and ICP automatically.

Join the Accelerator

Custom Business OS (done-for-you)

If you want us to build, deploy, and train your team on the full Business OS, we run a custom service. Best for teams of 5 to 50 where the founder's time is more expensive than the setup cost.

Book a call

Appendix D. FAQ

"Do I need to be a developer?"

No. Skill 1 is install Obsidian, run a skill, edit text files. Skill 2 and Skill 3 are run a skill and read its output. Skill 4 is one plugin swap. Skill 5 is one Railway account token, one click deploy. The whole stack is "install an app and edit a text file" with two five-minute exceptions.

"Will this replace Notion?"

For knowledge and AI-assisted work, yes. For project management with views, kanbans, and structured databases, no. Use both for different jobs. Most teams keep Notion for structured data (sales pipeline, customer database, content calendar) and use the vault for unstructured knowledge.

"Can I use Logseq or VS Code instead of Obsidian?"

Yes, both work. Logseq is more outliner-shaped and weaker on graph view. VS Code is more developer-shaped and weaker on writing UX. Obsidian is the strongest default for non-developers.

"What about iCloud sync? Can I just use that for the team?"

For one user across their own devices, iCloud works (with occasional sync conflicts). For a team, iCloud does not work: no real-time sync, no permissions, no deletion safety. You need Skill 4.

"How much does this cost per month?"

COMPONENT	COST
Obsidian	$0
Claude Pro	$20
Relay (small team free tier)	$0
Railway (vault MCP)	$0 to $25
Ben AI Accelerator (optional)	$XX/mo
Minimum total	**$20/mo**
Comfortable total	**$50/mo**

"Do I need all 5 skills?"

No. Skill 1 alone is valuable. Skill 1 + 2 + 3 is the personal OS most solo users will live in. Skill 4 only matters if you have teammates. Skill 5 only matters if you want autonomous runs in the cloud with your laptop closed. Use what you need.

"What if I already have a knowledge base in Notion / Roam / OneNote?"

Migrate it (see the migration section in Skill 1). Drop the markdown export into the vault and let `/os-optimizer` clean it up. Do not hand-clean before importing.

"What if Claude Cowork is not available in my country?"

Claude Code (the CLI) works anywhere Claude works. The whole guide works with Claude Code as the AI side. You lose nothing critical.

"Can I keep some folders private from my team?"

Yes. The BenAI Relay fork gives you folder-level permissions. `Departments/Finance/` invisible to anyone except finance. `Personal/` invisible to everyone except you. Whole files, not partial reads.

"What happens if the Operator writes something wrong?"

It posts a flag to `#vault-flags` and waits for review. The Operator's hard rules (Appendix B template) prevent it from writing to `Context/` without approval. First two weeks you read its outputs every morning. After that, you trust the routine ones and review the flagged ones.

"Is this open source?"

The pattern is. The skills, the Relay fork, and the Operator templates are Ben AI's. You can build all of it yourself by hand using only the public components (Obsidian, official Relay, Anthropic's `CLAUDE.md` best practices). The skills exist so you do not have to.

"What if I want to switch to a different vault tool later?"

Everything is plain markdown in a folder. You can move the folder anywhere. Open it in VS Code, Logseq, plain Finder. Wikilinks are an Obsidian-flavored markdown extension; other tools either support them or render them as plain text. The portability is the whole point.

"Can I run `/os-operator` and `/os-optimizer` autonomously without my laptop on?"

Yes, after Skill 5. The local versions of these skills run on your laptop while it is on. Skill 5 (`/os-mcp`) moves them to cloud routines that run with your laptop closed.

Appendix E. Glossary

Vault. A folder of markdown files that is your second brain. Same as "knowledge base." Same as "Business OS."

BenAI OS Plugin. The Claude plugin (named `obsidian` internally) that ships all 5 skills in this guide.

Cowork. Claude's desktop app for working on a folder of files. Reads and writes the vault.

Claude Code. Claude's CLI. Runs in a terminal. Reads and writes the vault.

`CLAUDE.md`. The bridge file at the vault root (and in every folder) that tells Claude how to behave and what lives where.

Plot.md / index.md. A folder's table of contents. `/os-optimizer` generates and refreshes these.

MCP (Model Context Protocol). The standard for giving Claude access to external tools and data. The OS MCP (Skill 5) wraps your Relay-synced vault into one interface.

Relay. Real-time CRDT sync plugin for Obsidian by System 3.

BenAI Relay fork. Our build of Relay that adds RBAC, folder-level permissions, and deletion safety. Installed by `/team-os`.

Relay MCP v2. The MCP server that talks to the Relay sync protocol so cloud agents can read and write your vault. Deployed by `/os-mcp` to your Railway account.

Skill. A small markdown file with instructions Claude can run as a command. Lives inside the plugin.

Routine. A scheduled task that touches the vault. Local routines need your laptop on. Cloud routines (via the OS MCP) do not.

Managed Agent. Anthropic's cloud-hosted agent runtime. Reads and writes through MCP servers.

Operator. An autonomous agent with a role file and recurring responsibilities. Owns the vault between humans touching it.

Champion. The human owner of the Operator. Maintains the role file. Approves changes. Drives adoption.

Lint. A pass that scans the vault for duplication, contradictions, dead links. Inspired by Karpathy's LLM Wiki. Runs as part of `/os-optimizer`.

Compounding context. The premise of the whole system: every note logged today makes the brain more useful tomorrow.

Final word

These 5 skills are the path from zero to a Team OS that runs itself.

If you only do Skill 1 today, you have already started. The compounding only works if you start.

If you get stuck, the Accelerator community is the fastest place to ask. We have been answering the same questions for months, so most of yours are already answered.

If you want us to do it for you, **book a call**.

Either way: start today. The 6 months of compounding context cannot be retroactively created.

Keep going,

Ben

Ben AI • benai.co • Version 3.0 • May 2026